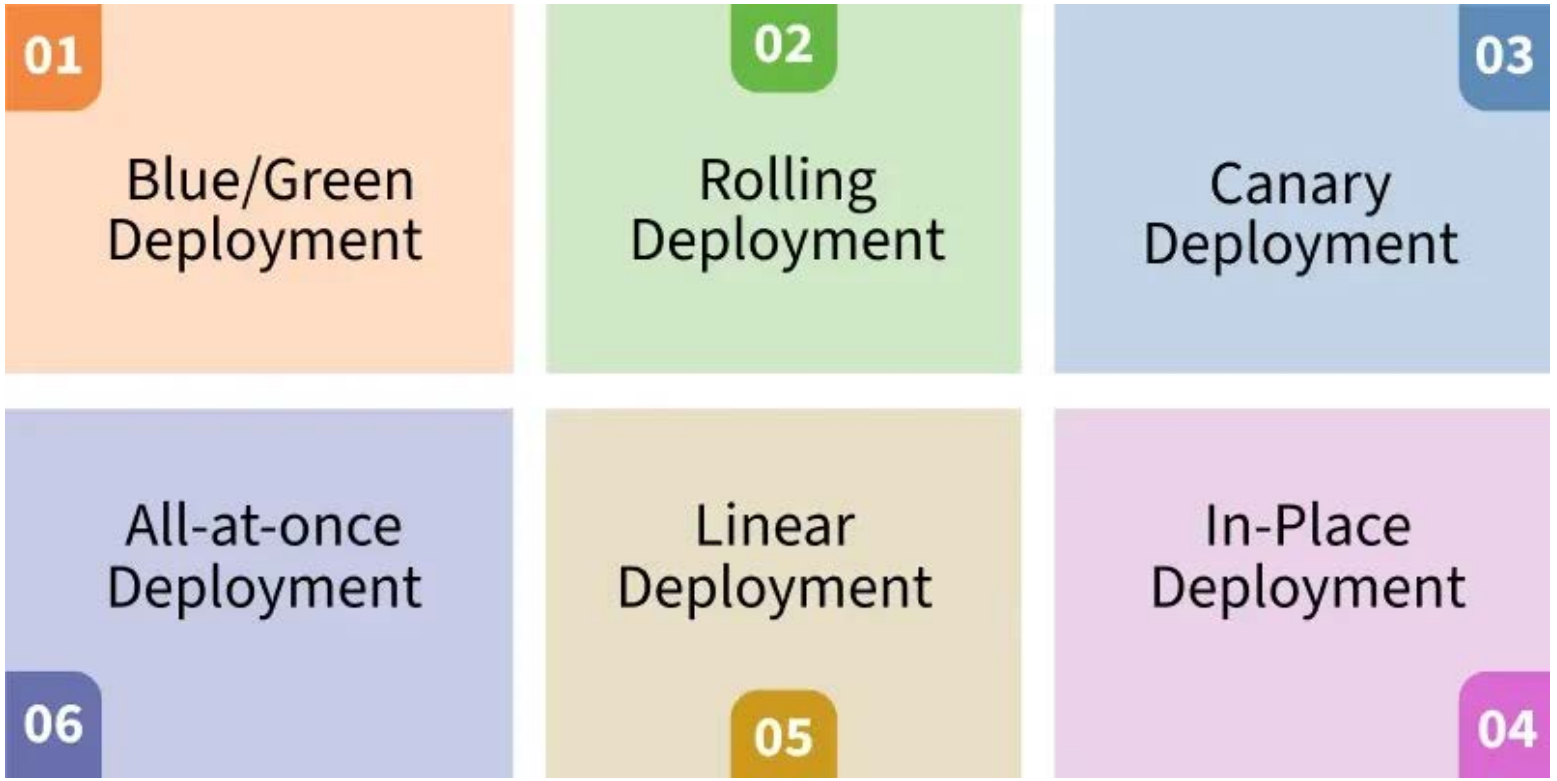
FULL-STACK APPLICATION DESIGN AND DEVELOPMENT

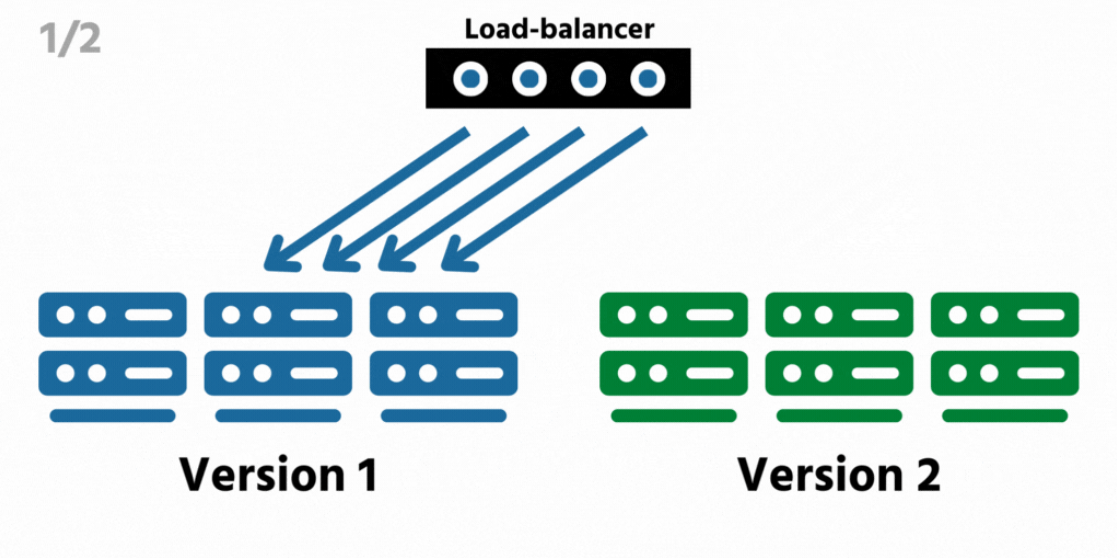
COURSE 14 - DEPLOYMENT STRATEGIES

DEPLOYMENT STRATEGIES

- Deployment strategies are important for managing application updates with minimal disturbance to users. Choosing the right deployment strategy is key for managing costs and reducing risks. These strategies are really helpful in reducing risks, optimizing resource use, and providing efficient transitions when deploying new versions of applications.



DEPLOYMENT STRATEGIES



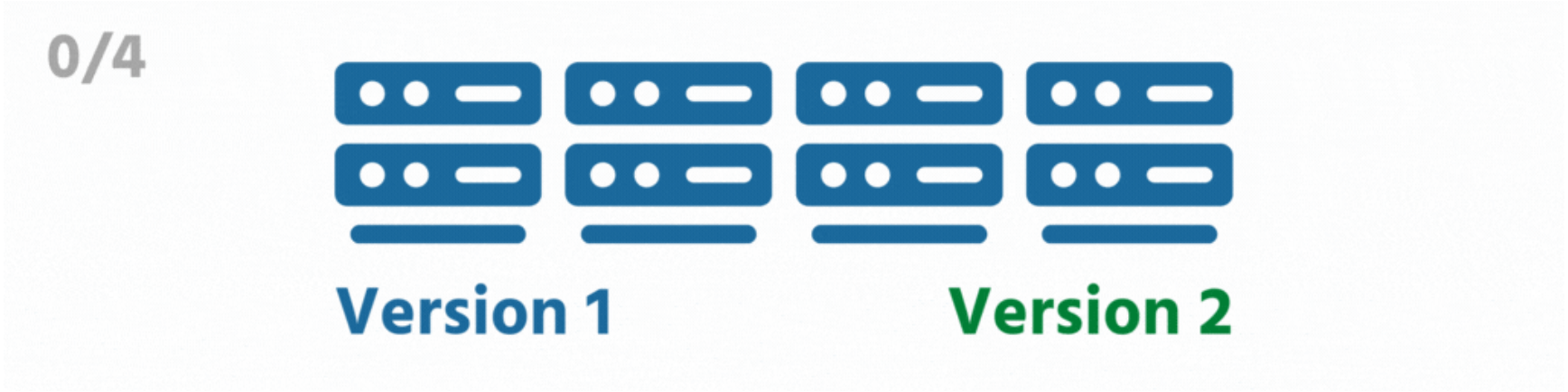
1. **Blue/Green Deployment**

- **Description**: In a blue/green deployment, you maintain two identical environments. The current environment (Blue) is live, while the new environment (Green) is used to deploy the updated application. After testing Green, traffic is switched from Blue to Green.
- **Example**: Imagine you have an e-commerce website running in the Blue environment. You deploy a new version in the Green environment (with a new checkout process). Once you confirm that Green works as expected, you switch all traffic to Green. If there are any issues, you can easily switch back to Blue.
- **Benefits**: Reduces downtime and ensures easy rollback.

DEPLOYMENT STRATEGIES

2. **Rolling Deployment**

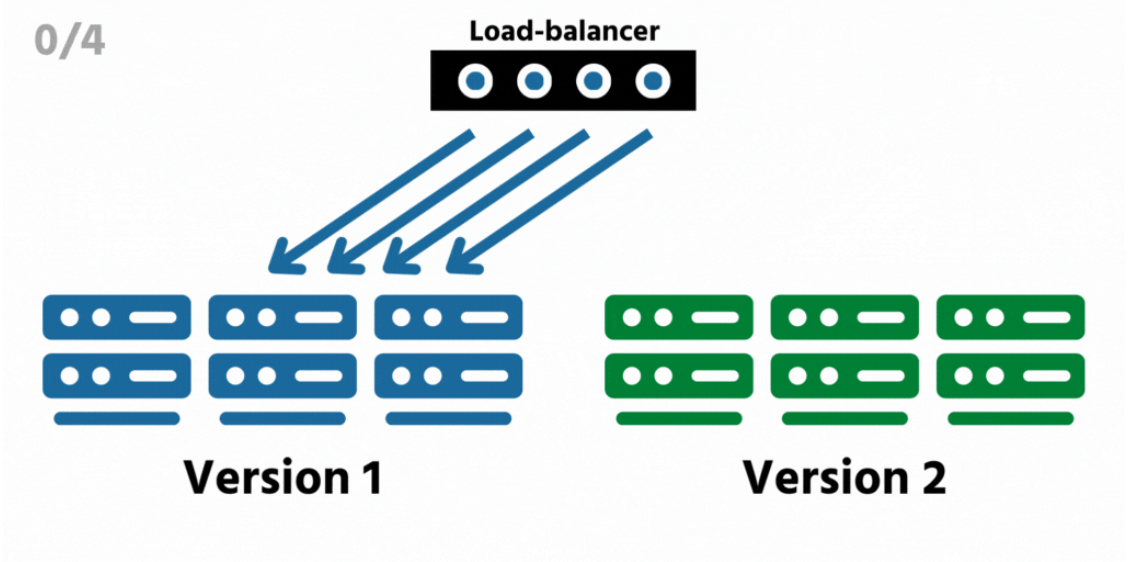
- **Description**: In this strategy, updates are rolled out to a few instances at a time. It ensures minimal disruption by gradually replacing old versions with new ones.
- **Example**: Suppose you have a fleet of web servers running a Content Management System (CMS). During a rolling deployment, only one server at a time will be updated with the new CMS version, while ensuring the rest of the servers remain operational, and users are still served.
- **Benefits**: Reduces downtime by ensuring some servers are always operational.



DEPLOYMENT STRATEGIES

3. **Canary Deployment**

- **Description**: A small percentage of traffic is initially sent to the new version of the app (the "canary"). If this version is stable, traffic is gradually increased until all users are served by the new version.
- **Example**: If you are deploying a new mobile app feature, you could start by directing only 5% of users to the new version. If everything runs smoothly, you can gradually increase the percentage of users to 100%.
- **Benefits**: Helps to identify issues early without affecting all users.



DEPLOYMENT STRATEGIES

4. **In-Place Deployment**

- **Description:** In an in-place deployment, the previous version of the application is stopped, while the new version is installed and started on the same server or computing resource. This approach ensures minimal infrastructure changes, as no new servers or systems are introduced.
- **Example:** Consider a web application hosted on a cloud server. With in-place deployment, you stop the web application, update its code, and restart it with the new version, all while using the same server infrastructure.
- **Benefits:**
  - **Lower cost**: No additional servers or infrastructure are required.
  - **Simplicity**: Easy to manage as it doesn't involve complex setups.
  - **Quick deployment**: Faster than setting up new servers or services for each deployment.

DEPLOYMENT STRATEGIES

5. **Linear Deployment**

- **Description:** In a linear deployment, traffic is shifted in equal increments, with a set number of minutes between each increment. The goal is to gradually move traffic from the old version to the new version, reducing the risk of failure by monitoring performance as traffic is shifted.
- **Example:** Suppose you have a web service and you want to migrate 100% of the traffic from version 1 to version 2. With linear deployment, you might shift 10% of the traffic every 10 minutes until all traffic is directed to version 2.
- **Benefits:**
  - **Lower risk**: Gradually shifting traffic allows you to catch potential issues early without affecting all users.
  - **Controlled environment**: Enables more fine-tuned control over the migration process.
  - **Flexibility**: You can adjust traffic increment size and time between shifts.

DEPLOYMENT STRATEGIES

6. **All-at-once ("Big Bang") Deployment**

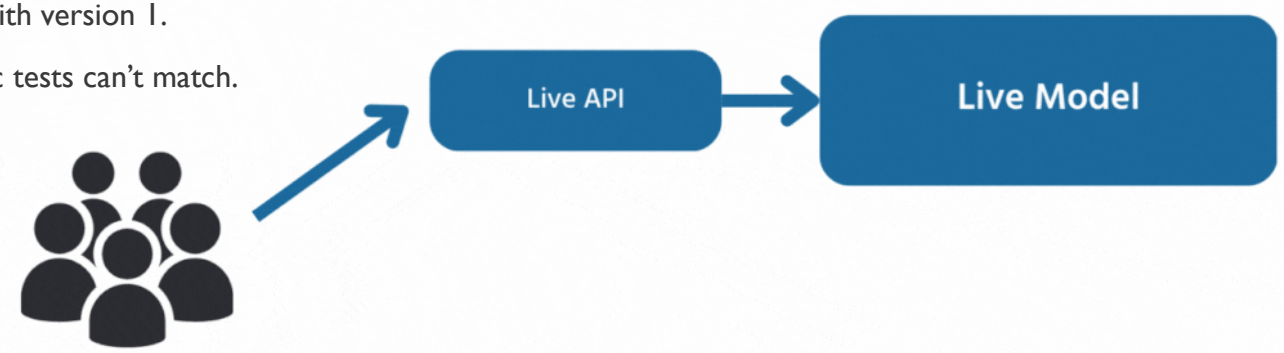
- **Description:** In an all-at-once deployment, all traffic is shifted from the original environment to the new environment instantly. This approach minimizes the deployment time but carries a higher risk since everything is switched over in a single step.
- **Example:** If you are updating a cloud-based application, all users are immediately routed to the new version after the deployment. There is no phased approach; the switch happens at once.
- **Benefits:**
  - **Faster execution**: The deployment happens in one go, saving time compared to gradual shifts.
  - **Simplicity**: Less configuration is needed as there are no increments or gradual adjustments.
  - **Easy to implement**: A straightforward method that is simple to manage.

DEPLOYMENT STRATEGIES

Other deployment strategies:

1. **Shadow Deployment / Dark Launching**

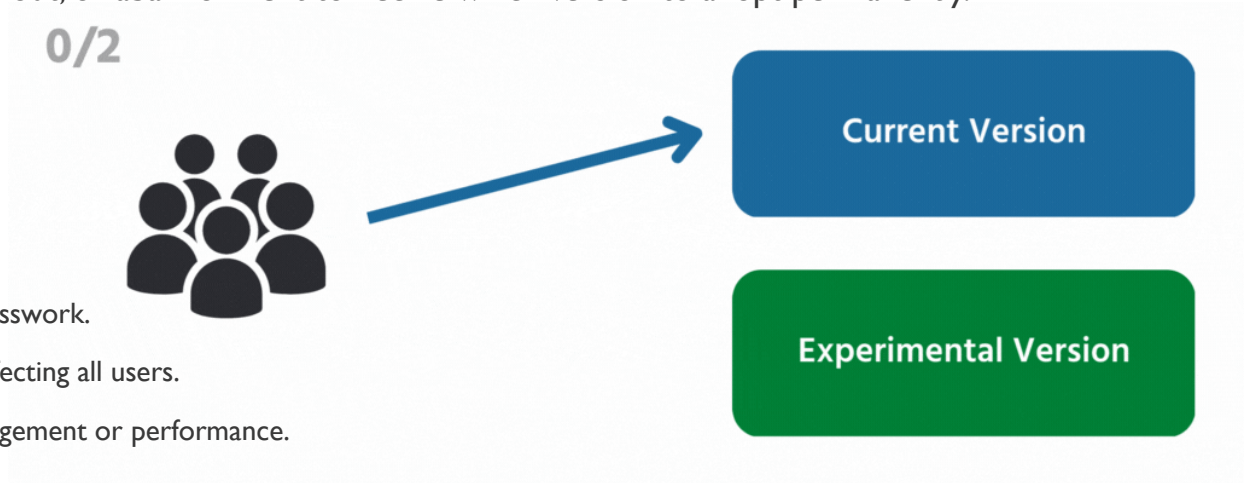
- **Description:** In a shadow deployment, the new version of the application runs **alongside** the current production version. It receives a **copy of real user traffic**, but its responses are **not returned to users**. This lets teams evaluate performance, stability, and behavior under real-world load without exposing users to risks.
- **Example:** You launch version 2 of a backend service in a shadow environment. When users make requests, version 1 handles them normally, but the same requests are also sent to version 2 "in the shadows". You log version 2's responses and performance metrics, but only version 1's responses are delivered to users.
- **Benefits:**
  - **Zero user impact:** Even if version 2 fails, users continue interacting with version 1.
  - **Real traffic testing:** Provides realistic performance data that synthetic tests can't match.
  - **Early failure detection:** Helps catch issues before any real rollout.



DEPLOYMENT STRATEGIES

2. **A/B Testing Deployment**

- **Description:** In A/B testing deployment, two different versions of an application (A and B) are served to **distinct groups of users** at the same time. The goal is not risk reduction but **comparing user behavior**, performance, and business outcomes across the versions to determine which performs better.
- **Example:** Version A of an e-commerce checkout page shows a traditional form, while version B introduces a simplified one-click checkout. 50% of users see A and 50% see B. You measure metrics like conversion rate, time to checkout, or abandonment to decide which version to adopt permanently.



- **Benefits:**
  - **Data-driven decisions:** Choose versions based on real user behavior, not guesswork.
  - **Product experimentation:** Safely test new features or UI changes without affecting all users.
  - **Improved user experience:** Helps identify which version delivers better engagement or performance.

DEPLOYMENT STRATEGIES

- **How to choose the right deployment strategy for your software project?**
- Choosing the right deployment strategy for your software project is all about finding the best fit for your specific needs. Here are a few things to keep in mind:
- **Size and complexity of the project -** If your project is on the smaller side and doesn't have too many dependencies, you might be able to get away with a simpler strategy like blue-green deployment or rolling deployment. But if you're working on something larger and more complex, a more advanced strategy like canary deployment or A/B testing deployment might be more your style.
- **Availability & scalability requirements -** Making sure your application is available and can handle a lot of traffic is important, especially in a production environment. If you need to be able to update your app quickly or roll back in case of issues, you'll want to look at strategies that allow for rolling updates.
- **Resources available -** Your team's resources will also play a role in choosing the right deployment strategy. If you're working with a small team or people who aren't as familiar with advanced strategies, you might want to stick with something simpler that's easy to set up and maintain.
- **Impact on end-users -** It's important to keep in mind how the deployment strategy will affect the end-users. Some strategies like Big-Bang deployment can cause a longer downtime, so it's important to choose a strategy that allows for minimal downtime.

DEPLOYMENT STRATEGIES

- **Deployment Best Practices**
- Application teams can follow a number of best practices to keep deployment risks to a minimum:
- **Use a deployment checklist.** For example, an item on the checklist may be to "backup all databases only after app services have been stopped" to prevent data corruption.
- **Adopt Continuous Integration (CI).** CI ensures code checked into the feature branch of a code repository merges with its main branch only *after* it has gone through a series of dependency checks, unit and integration tests, and a successful build. If there are errors along the path, the build fails and the app team is notified. Using CI therefore means every change to the application is tested before it can be deployed. Examples of CI tools include: CircleCI, Jenkins.
- **Adopt Continuous Delivery (CD).** With CD, the CI-built code artifact is packaged and always ready to be deployed in one or more environments.
- **Use standard operating environments (SOEs)** to ensure environment consistency. You can use tools like Vagrant and Packer for development workstations and servers.

DEPLOYMENT STRATEGIES

- **Use Build Automation tools.** With these tools, it's often simple to click a button to tear down an entire infrastructure stack and rebuild from scratch. An example of such tools is CloudFormation.
- **Use configuration management tools** like Puppet, Chef, or Ansible in target servers to automatically apply OS settings, apply patches, or install software.
- **Use communication channels** like Slack for automated notifications of unsuccessful builds and application failures.
- **Create a process for alerting the responsible team on deployments that fail.** Ideally you'll catch these in the CI environment, but if the changes are deployed you'll need a way to notify the responsible team.
- **Enable automated rollbacks for deployments** that fail health checks, whether due to availability or error rate issues.

DEPLOYMENT STRATEGIES

- **Post-Deployment Monitoring**
- Even after you adopt all of those best practices, things may still fall through the cracks. Because of that, monitoring for issues occurring immediately after a deployment is as important as planning and executing a perfect deployment.
- An application performance monitoring (APM) tool can help your team monitor critical performance metrics including server response times after deployments. Changes in application or system architecture can dramatically affect application performance.
- An error-monitoring solution like Rollbar is equally essential. It will quickly notify your team of new or reactivated errors from a deployment that could uncover important bugs requiring immediate attention.
- Without an error monitoring tool, the bugs may never have been discovered. While a few users who encounter the bugs will take the time to report them, most others don't. The negative customer experience can lead to satisfaction issues over time, or worse, prevent business transactions from taking place.
- An error monitoring tool also creates a shared visibility of all the post-deployment issues among Operations / DevOps teams and developers. This shared understanding allows the teams to be more collaborative and responsive.